

Sentinelleai — Audit Report

MagnetaOFTAutoLiquidityFactory.sol · afad9e65-6f14-4ab3-b39d-626f7b3530d7 ·
2026-05-22 15 h 55 min 50 s

CAUTION 72 / 100 6 consolidated findings

MagnetaOFTAutoLiquidityFactory presents a moderate risk profile. The most consistently confirmed finding (ZAD-1, SC01) is that `setCrossChainCreator` lacks a zero-address guard, confirmed by the static sentinel, developer, attacker, historian, and formalist agents — though the impact is limited to operational disruption rather than fund loss. The static sentinel's FACT-2 finding is explicitly refuted by four agents: the code at line 92 already contains `require(creator != address(0))`. The formalist agent's MEDIUM finding on FACT-2 and HIGH finding on upgradeability are rejected — the former is contradicted by the source, and the latter is inapplicable as the contract is not upgradeable (no UUPS/proxy pattern). The attacker agent raises valid centralization concerns (SC01) around single-owner control of treasury and `crossChainCreator` with no timelock, which is a real governance risk for high-value deployments. Registry spam (SC10) and replay/idempotency concerns (SC03) are noted but low-confidence without visibility into the cross-chain module. No reentrancy, arithmetic, or critical fund-loss vulnerabilities were identified. Overall the contract is functional but carries centralization and minor input-validation gaps warranting a CAUTION verdict.

Severity breakdown

1 MEDIUM 3 LOW 2 INFO

Static sentinel pre-scan

Static sentinels matched 2 pattern(s) — 1 MEDIUM, 1 LOW.

LOW SC01 · ZAD-1 · line 46

Setter accepts zero-address (no validation)

set*(address _x) functions that write the parameter directly to state can brick the protocol if called with address(0). Add require(_x != address(0)).

```
function setCrossChainCreator(address _creator) external onlyOwner { emit  
CrossChainCreatorUpdated(crossChainCreator, _creator); crossChainCreator = _creator;  
}
```

MEDIUM SC01 · FACT-2 · line 83

Factory accepts address parameters without zero-address check

create*/deploy* functions that store `address` parameters (token, recipient, manager) without `require(\_x != address(0))` brick the resulting pool — token transfers revert, fees go to the zero address, etc.

```
function createForCreator – unchecked: creator
```

Consolidated findings

MEDIUM CVSS 5.5 SC01 - Access Control

Single-owner control over treasury and crossChainCreator with no timelock or role separation

The factory owner can immediately change both treasury and crossChainCreator with no enforced timelock, multisig requirement, or role separation. A compromised or malicious owner EOA can redirect the treasury address for all subsequently created tokens (affecting transfer-tax flows in the token template) and can set crossChainCreator to an attacker-controlled contract to register tokens under arbitrary creators. This is a centralization risk consistent with OWASP SC01 private-key-compromise patterns.

```
setTreasury(address _newTreasury), setCrossChainCreator(address _creator)
```

→ Require the owner to be a multisig or governance contract. Add a timelock of at least 24 hours for setTreasury and setCrossChainCreator. Consider splitting treasury-management and cross-chain-module roles. Emit queued changes before execution and monitor for unauthorized updates.

Confirmed by: attacker

LOW CVSS 3.5 SC01 - Access Control / Input Validation

ZAD-1: setCrossChainCreator accepts zero address without validation

The setCrossChainCreator function writes _creator directly to state without a zero-address check. Setting crossChainCreator to address(0) bricks the createForCreator path because the guard condition (msg.sender != crossChainCreator || crossChainCreator == address(0)) will always revert. While this may be an intentional disable mechanism, it is undocumented and can cause an operational outage if called by mistake or by a temporarily compromised owner.

```
MagnetaOFTAutoLiquidityFactory.sol, line 46: setCrossChainCreator(address _creator)
```

→ Add require(_creator != address(0), 'CrossChainCreator cannot be zero address') before the assignment, or introduce a separate explicit disableCrossChainCreator() function with clear documentation of its effect.

Confirmed by: static-sentinel, developer, attacker, historian, formalist

LOW CVSS 3.1 SC10 - Denial of Service

Unbounded allTokens and userTokens arrays enable registry spam and potential DoS

createOFTAutoLiquidityToken is free except for gas and appends every created token to the unbounded allTokens array and the caller's userTokens array. An attacker can repeatedly call this function to bloat these arrays. getUserTokens returns the full dynamic array in memory, which can become impractical or revert for heavily spammed addresses. Off-chain indexers enumerating allTokens can become slow or time out.

```
createOFTAutoLiquidityToken(...), createForCreator(...),  
getUserTokens(address), allTokens array
```

→ Add pagination APIs for user tokens and global tokens. Avoid returning unbounded arrays in view functions. Consider creation fees, storage bonds, or rate limits to deter spam. Monitor deployment volume.

Confirmed by: attacker

LOW CVSS 2.8 SC03 - Logic Errors

No cross-chain replay or idempotency protection in createForCreator

createForCreator authenticates only msg.sender == crossChainCreator and does not record a nonce, GUID, source chain ID, or request hash. If the cross-chain creator module is misconfigured or lacks replay tracking, the factory will accept repeated module calls and deploy duplicate tokens for the same source-chain request, registering all duplicates in userTokens and allTokens. The factory delegates all replay safety to an external module without enforcing idempotency at the last trusted boundary.

```
createForCreator(...)
```

→ Include a bytes32 requestId or LayerZero GUID/source-chain/nonce tuple in createForCreator, store usedRequests[requestId] = true, and revert on duplicates. Ensure the cross-chain module enforces peer trust and message replay tracking.

Confirmed by: attacker

INFO CVSS 0.0 SC01 - Input Validation

FACT-2 Refuted: createForCreator already validates creator is not zero address

The static sentinel flagged createForCreator as missing a zero-address check on the creator parameter. The source code at line 92 explicitly contains require(creator != address(0), 'Creator cannot be zero') before token deployment and registry writes. This finding is a false positive from the static sentinel.

```
MagnetaOFTAutoLiquidityFactory.sol, line 92: createForCreator
```

→ No action required. Maintain existing validation and add regression tests confirming createForCreator(address(0), ...) reverts.

Confirmed by: developer, attacker, historian

INFO CVSS 0.0 SC07 - Upgradeable Contract Vulnerabilities

Formalist upgradeability finding rejected — contract is not upgradeable

The formalist agent raised a HIGH finding about `_authorizeUpgrade` requiring multisig. This is inapplicable: `MagnetaOFTAutoLiquidityFactory` does not implement `UUPS`, `TransparentProxy`, or any upgradeable pattern. It inherits only `Ownable2Step` and `ReentrancyGuard`. No `_authorizeUpgrade` function exists in this contract.

`MagnetaOFTAutoLiquidityFactory.sol` (entire contract)

→ No action required. Finding is not applicable to this contract.

Historical precedents

Retrieved from the local bug-bounty corpus (~20 700 findings) by vulnerability-class match.

CRITICAL `access-control` ethereum 2025-05-28 **\$12 000 000**

Corkprotocol — Access Control (2025-05-28)

https://x.com/SlowMist_Team/status/1928100756156194955

HIGH `access-control` bsc 2023-03-28 **\$8 900 000**

SafeMoon Hack — Access Control (2023-03-28)

https://twitter.com/zokyo_io/status/1641014520041840640

HIGH `access-control` linea 2024-06-01 **\$6 880 000**

VeloCore — lack-of-access-control (2024-06-01)

<https://x.com/BeosinAlert/status/1797247874528645333>

LOW `bm25` 2021-11-01

nested — Missing zero-address check in setters

<https://github.com/code-423n4/2021-11-nested-findings/issues/83>

MEDIUM `bm25` 2023-07-01

tapioca — MISSING ZERO-ADDRESS CHECK IN CONSTRUCTOR

<https://github.com/code-423n4/2023-07-tapioca-findings/issues/81>

GAS `bm25` 2021-11-01

nested — Add zero-address checkers

<https://github.com/code-423n4/2021-11-nested-findings/issues/108>

MEDIUM `bm25` 2023-04-01

frankencoin — `\_challengeSeconds` accepts 0 value when creating a position

<https://github.com/code-423n4/2023-04-frankencoin-findings/issues/794>

MEDIUM `bm25` 2022-05-01

rubicon — `RubiconMarket.feeTo` set to zero-address can DoS `buy` function

<https://github.com/code-423n4/2022-05-rubicon-findings/issues/319>

MEDIUM `bm25` 2023-04-01

rubicon — `feeTo` set to zero-address can DoS the `rubical1` function

<https://github.com/code-423n4/2023-04-rubicon-findings/issues/213>

MEDIUM `bm25` 2024-07-01

loopfi — Not all entry points calling PendleRouter::addLiquiditySingleToken() accepts native tokens

<https://github.com/code-423n4/2024-07-loopfi-findings/issues/208>

MEDIUM bm25 2023-09-01

ondo — mutiexcall accepts unbounded array of call data as parameter, the general function should have more validations.

<https://github.com/code-423n4/2023-09-ondo-findings/issues/41>

MEDIUM bm25 2023-09-01

ondo — mutiexcall in rUSDYFactory accepts unbounded array of call data as parameter, the general function should have more validations.

<https://github.com/code-423n4/2023-09-ondo-findings/issues/79>

Generated by Sentinelleai on 2026-05-22T20:13:39.545Z. Audit ID: afad9e65-6f14-4ab3-b39d-626f7b3530d7.

This report combines deterministic regex+AST sentinels, optional Slither/Mythril static analysis, and a 6-model adversarial LLM panel consolidated by a Judge agent.